Software Engineering

Dr. Fatma ElSayed

Dr. Ahmed Yousry

Computer Science Department

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part 1
- System Modelling Part 2
- System Modelling Part 3
- Software Testing Part 1
- Software Testing Part 2
- System Architecture

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part 1
- System Modelling Part 2
- System Modelling Part 3
- Software Testing Part 1
- Software Testing Part 2
- System Architecture

Chapter 8: Software Testing

Software Testing

- Software testing is the process of identifying the **completeness, correctness, and quality of the developed software**.
- Testing is the process of executing program or application with the **intent of finding errors.**
  - To demonstrate to the developer and the customer that the software meets its requirements.
  - To discover situations in which the behavior of the software is incorrect, undesirable, or does not conform to its specification.

Who Should Test?

- To be **most effective**, testing should be conducted by **an independent third party.**

Verification

- "Are we building the product right"
- During development
- Involves checking documents provided by humans.
- Process-oriented
- Does not require code execution, so it is considered **static testing**

Validation

- "Are we building the right product"
- After development, pre-release
- Involves the execution of a program by a computer
- Product-oriented
- Requires code execution, so it is considered **dynamic testing**

Testing Stages

- A commercial software system has to go through **three** stages of testing:

1. **Development testing**
   - Where the system is tested during development to discover bugs and defects. System **designers and programmers** are likely to be involved in the testing process.
2. **Release testing**
   - Where a **separate testing team** tests a complete version of the system before it is released to users. The aim of release testing is to check that the system meets the requirements of system stakeholders.
3. **User testing**
   - Where users or potential users of a system test the system in their own environment. For software products, the 'user' may be an internal marketing group who decide if the software can be marketed, released, and sold.

Testing Stages

- Development testing
  - Levels
    - Unit testing
    - Integration testing
    - System testing
- Release testing
- User testing

Testing Stages

- Development testing
- Release testing
- User testing

Levels

- Unit testing
- Integration testing
- System testing

Unit Testing Considerations

- Where **individual program units** or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
- The module **interface** is tested to ensure that information properly flows into and out of the program unit under test. Tests of data flow across a module interface **are required before** any other test is initiated.

Interface Testing Involve:

- Input Validation
- Output Validation
- Error Handling

Ex: Suppose you're testing a login module:

Input: Username and password.

Output: A success message or error.

Interface testing would ensure:

Username and password are received correctly.

Errors are properly returned for invalid inputs.

The output format matches expectations.

Unit Testing Considerations

- Among the more common errors in **computation** are:
  - Misunderstood or incorrect arithmetic precedence
  - Incorrect initialization
  - Incorrect symbolic representation of an expression.
  - Boundary conditions
- **Comparison** *(change of flow frequently occurs after a comparison)*, test cases should uncover errors such as:
  - Comparison of different data types
  - Incorrect logical operators or precedence
  - Improper loop termination or modified loop variables

Testing Methodology



- Full Knowledge
- Testing as a Developer
- Used to check that the details of an implementation are correct.



- Zero Knowledge
- Testing as a User
- Used to test that each of the customer's requirements is handled correctly

White Box Testing Techniques

Basis Path Testing

- It is a white-box testing technique based on the **control structure** of a program or a module.
- All **independent paths (basis paths)** through the control structure are exercised to ensure that all statements in a module have been executed at least once.
- To design test cases using this technique, four steps are followed:
  1. Construct the Control Flow Graph(CFG)
  2. Compute the Cyclomatic Complexity Metric(CCM) of the Graph
  3. Identify the Independent Paths
  4. Design Test cases from Independent Paths

White Box Testing Techniques

Basis Path Testing

(1)	(2)	(3)	(4)
Construct the Control Flow Graph(CFG)	Compute the Cyclomatic Complexity Metric(CCM) of the Graph	Identify the Independent Paths	Design Test cases from Independent Paths

White Box Testing Techniques

Basis Path Testing

(1)

Construct the Control Flow Graph(CFG)

Flow Graph Notation

- Control Flow Graph is represented differently for all statements and loops.

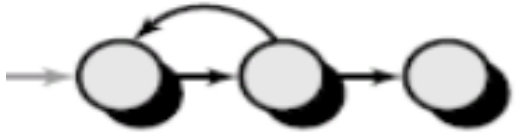
Sequence



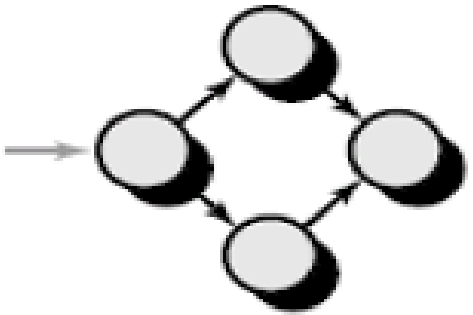
While



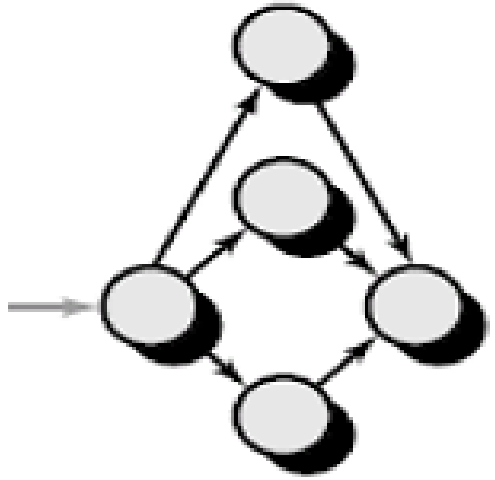
Do-while



if



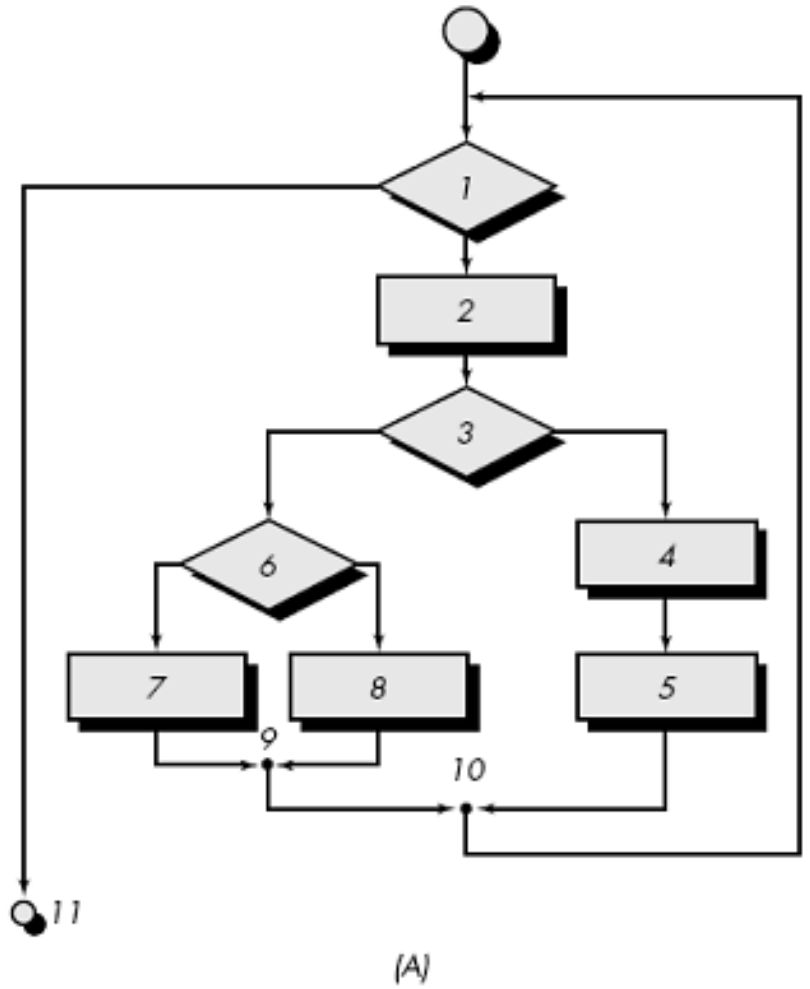
Case



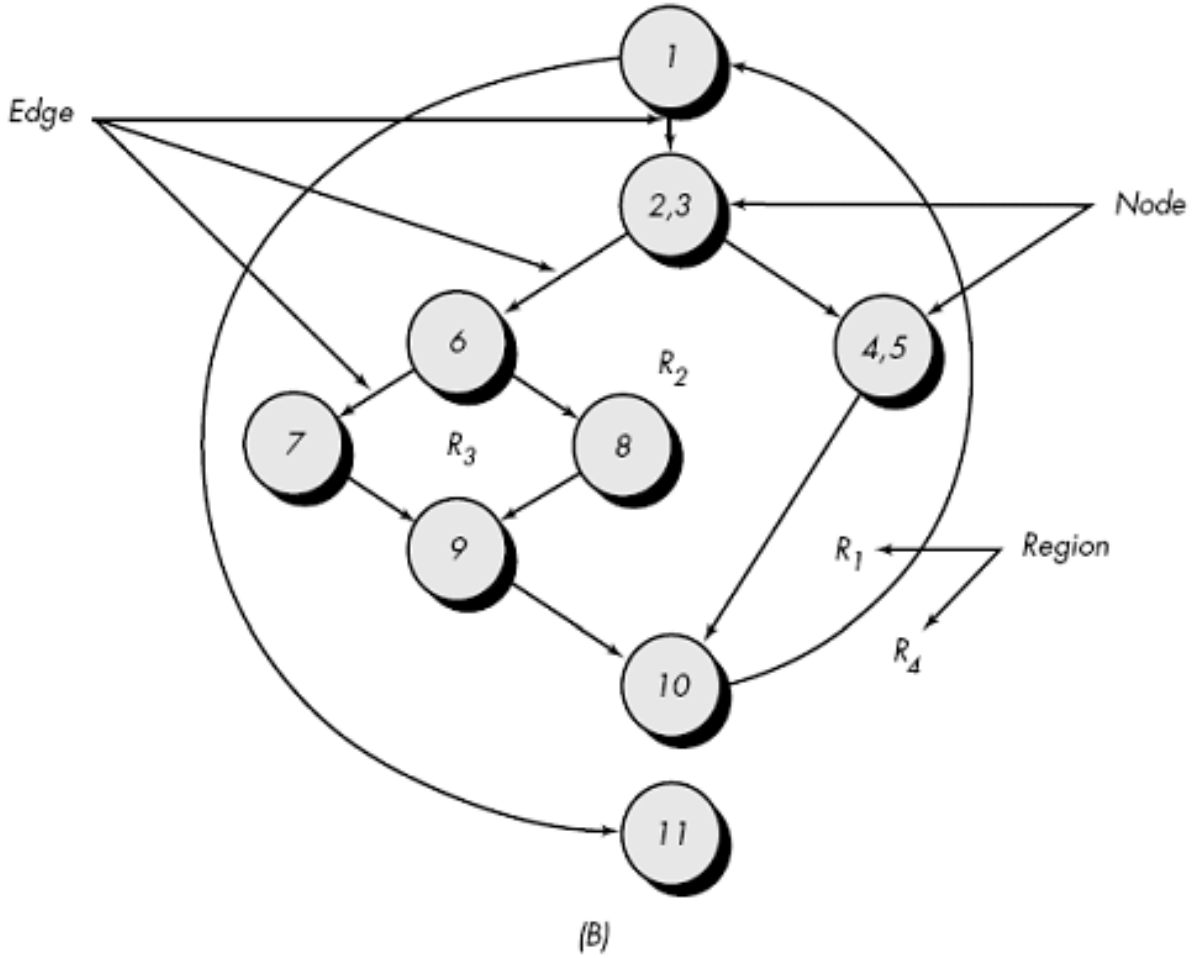
Flow Graph

- Each **circle**, called a flow graph **node**, represents one or more procedural statements.
- A **sequence** of process boxes and a decision diamond can map into a **single** node.
- The **arrows** on the flow graph, called **edges** or links, represent a flow of control.
- Areas bounded by edges and nodes are called **regions**. When counting regions, *we include the area outside the graph as a Region.*
- Each node that contains a **condition** is called a **predicate node** and is characterized by two or more edges come out from it.

Flow Chart vs Flow Graph



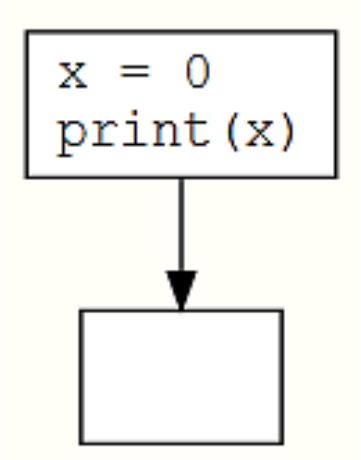
(A)

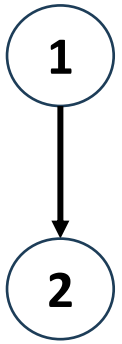


(B)

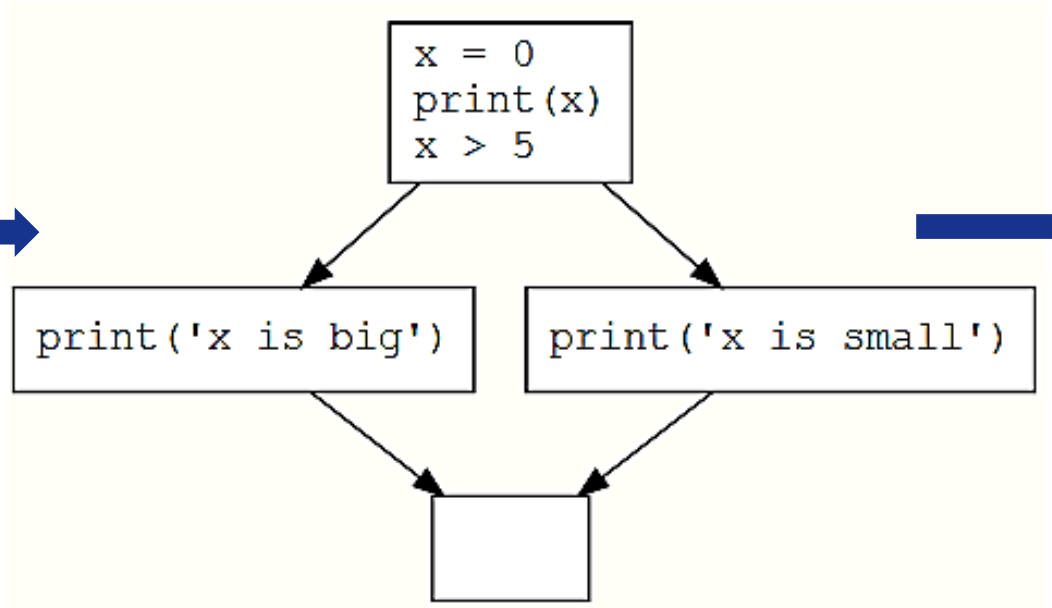
Flow Graph

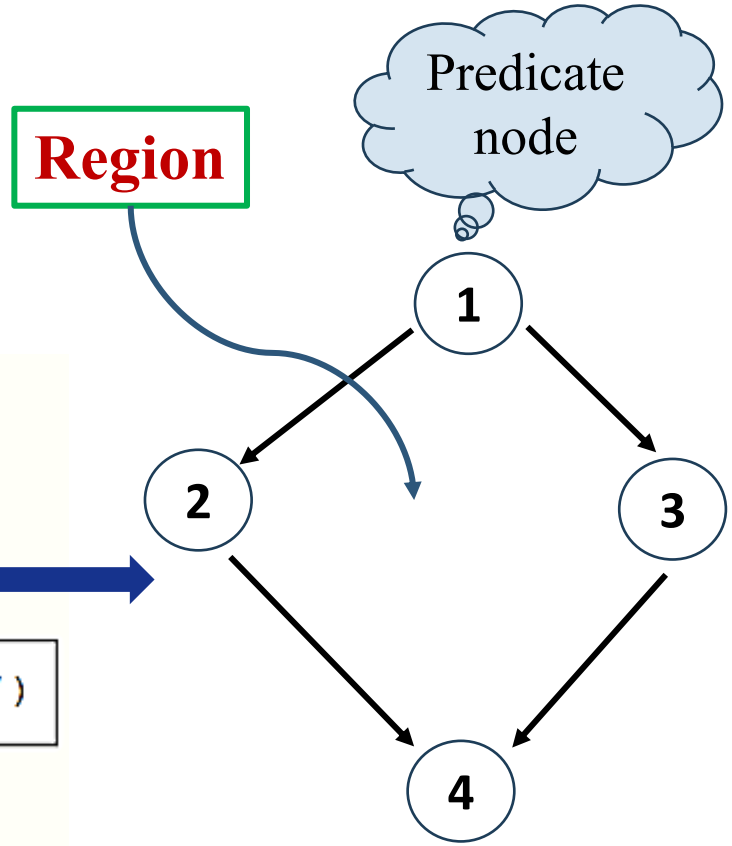
```
x = 0
print(x)
```





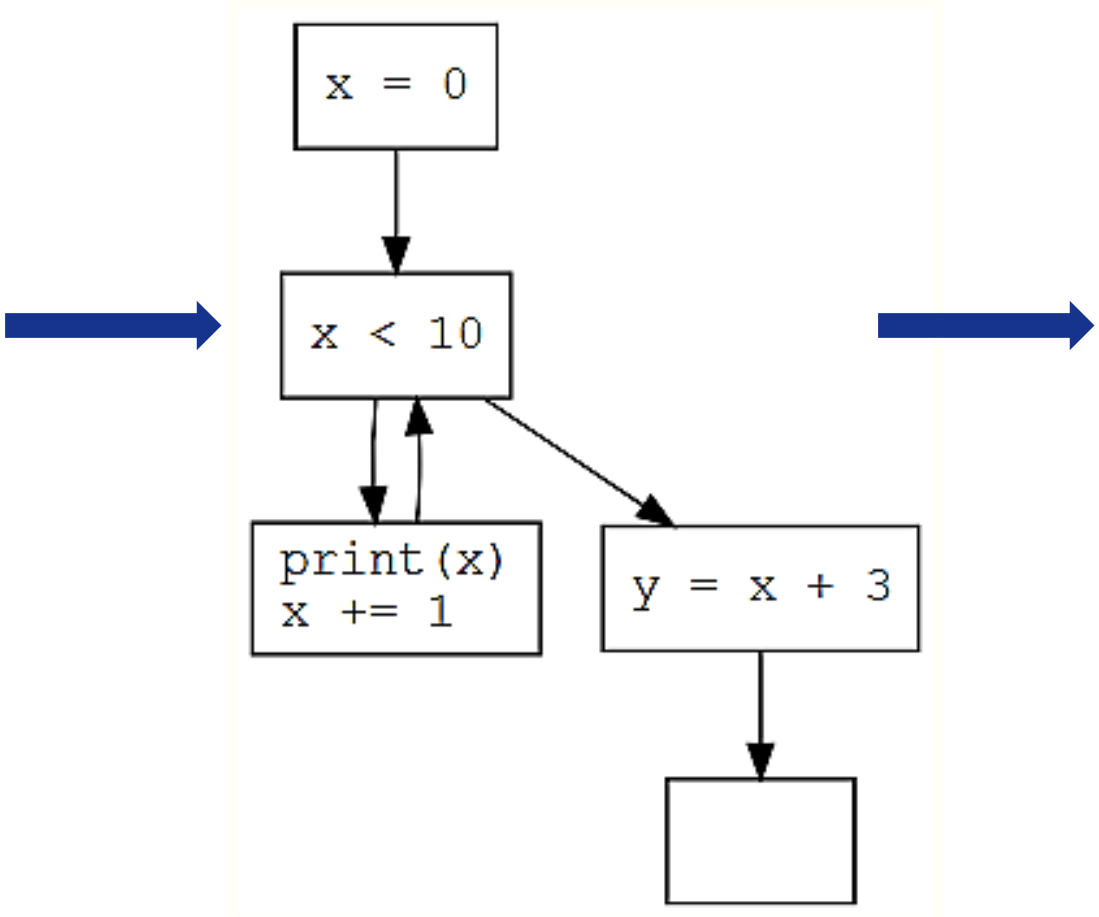

```
x = 0
print(x)
if (x > 5)
    print('x is big')
else:
    print('x is small')
```

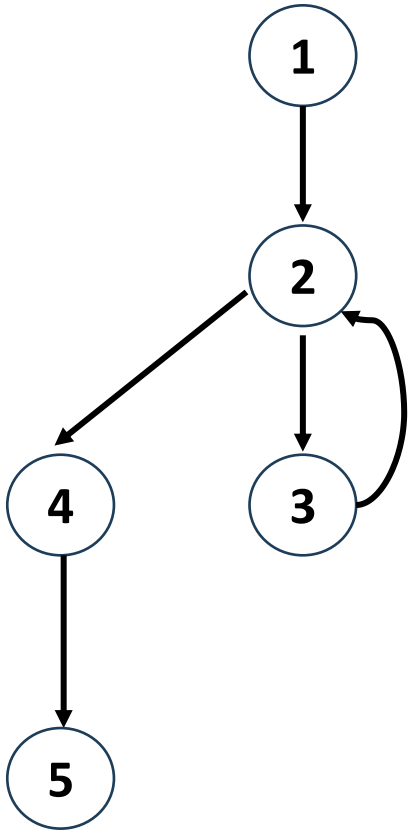




Flow Graph

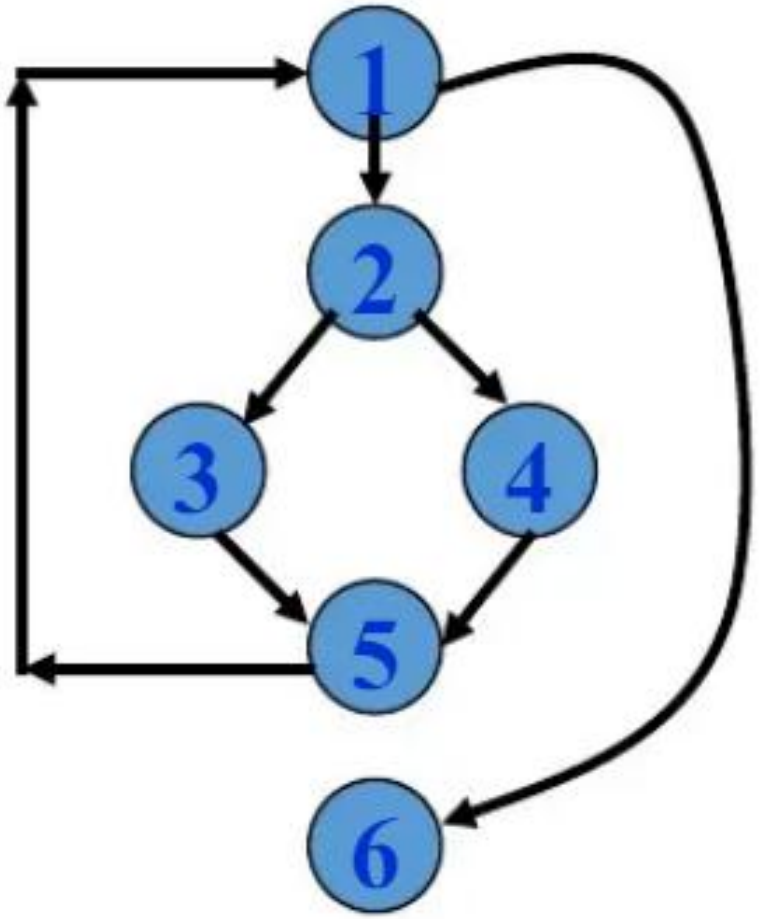
```
void fun(){
    x = 0
    while (x < 10){
        print(x)
        x += 1
    }
y = x + 3
}
```





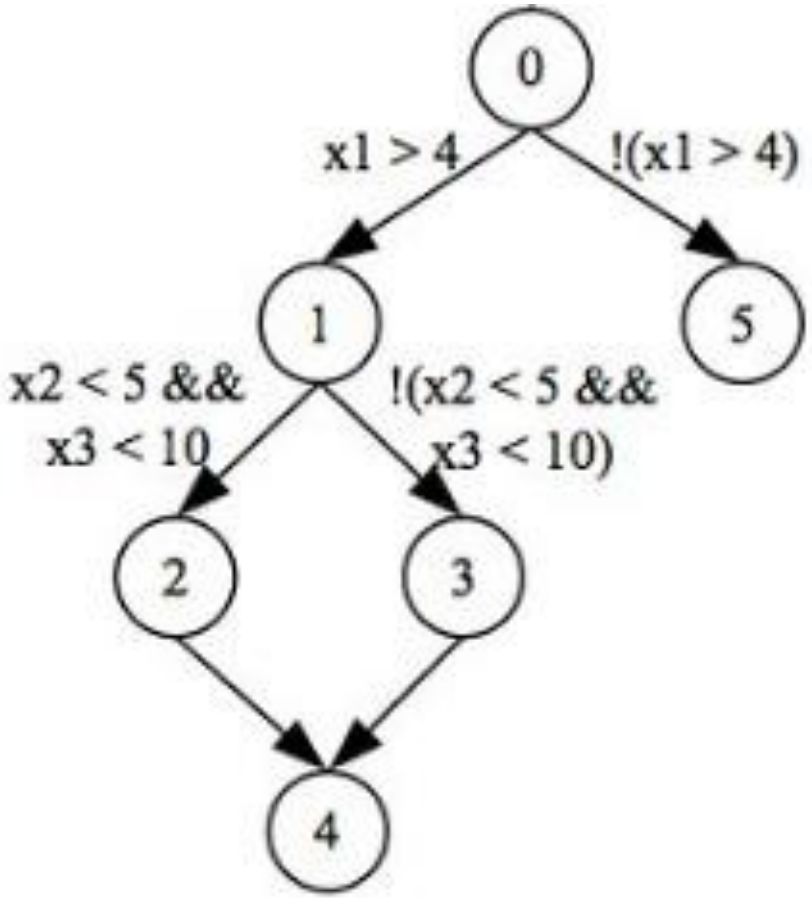
Flow Graph

```
int f1(int x, int y){
    while(x != y){
        if(x>y)
            x=x-y;
        else
            y=y-x;
        }
    return x;
}
```



Flow Graph Notation

```
void function1(int x1, int x2, int x3)
{
0  int value = 0;
   if(x1 > 4)
   {
1    if(x2 < 5 && x3 < 10)
     {
2      value = func2(x2, x3);
       cout << "func2 = " << value;
     }
     else
     {
3      value = func3(x2, x3);
       cout << "func3 =  " << value;
     }
4    return value * 2;
   }
   else
   {
5    value = func4(x1);
     return value;
   }
}
```



White Box Testing Techniques

Basis Path Testing

(1)

Construct the Control Flow Graph(CFG)

(2)

Compute the Cyclomatic Complexity Metric(CCM) of the Graph

Cyclomatic Complexity

- It is a software metrics that provides a **quantitative** measure of the logical complexity of a program.
- It defines **no. of independent paths** in the basis set, and
- Provides an **upper** bound for the **no. of tests** that must be conducted.

Cyclomatic Complexity

- **Three Ways to Measure**



Graph Theory

$$V(G) = E - N + 2$$

Calculates based on structural elements (Edges and Nodes).



Regions

$$V(G) = R$$

Calculates based on enclosed areas plus the exterior.



Predicates

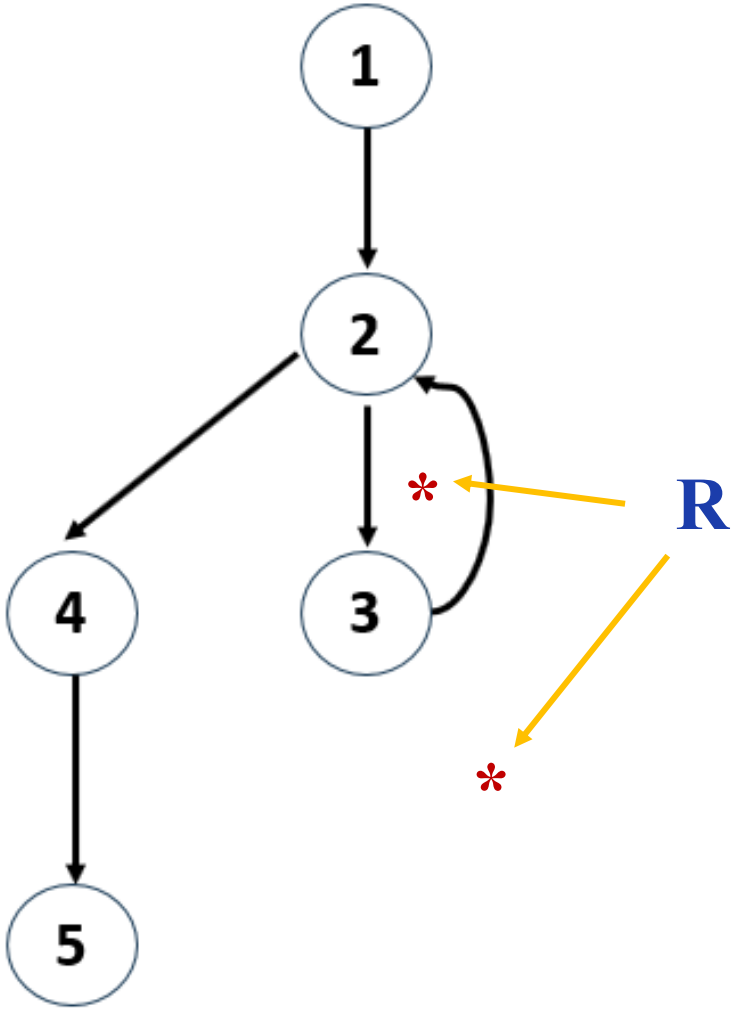
$$V(G) = P + 1$$

Calculates based on decision points (If, While, Case).

Cyclomatic Complexity

Cyclomatic Complexity

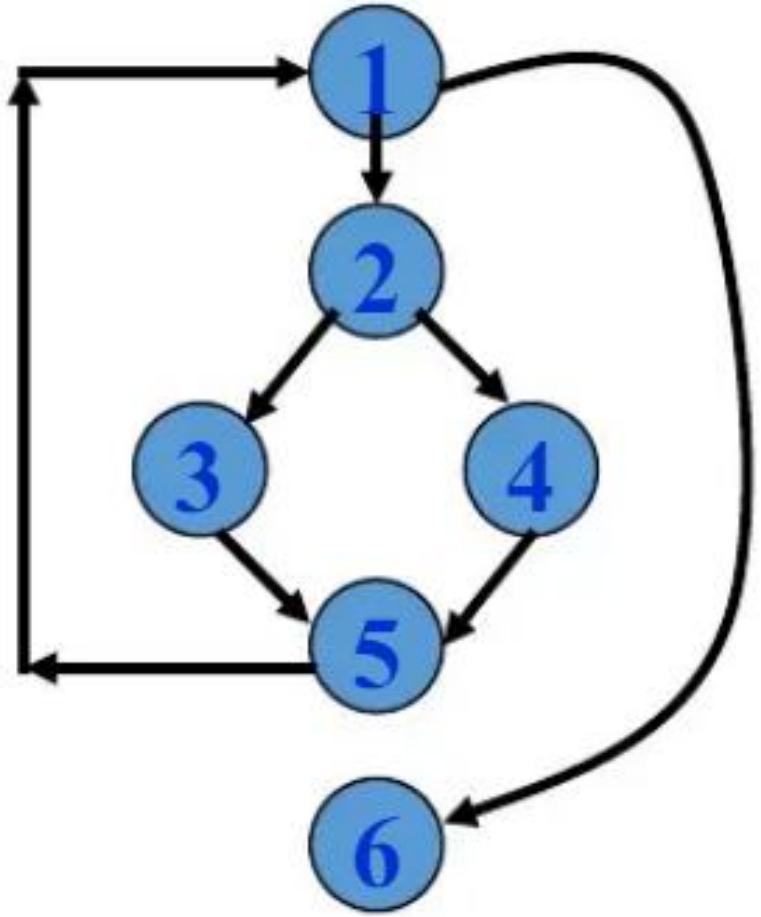
1. V(G) = R, the flow graph has **2** regions.
2. V(G) = E-N+2 = 5 edges – 5 nodes +2 = **2**.
3. V(G) = P+1 = 1 predicate nodes + 1 = **2**



Cyclomatic Complexity

Cyclomatic Complexity

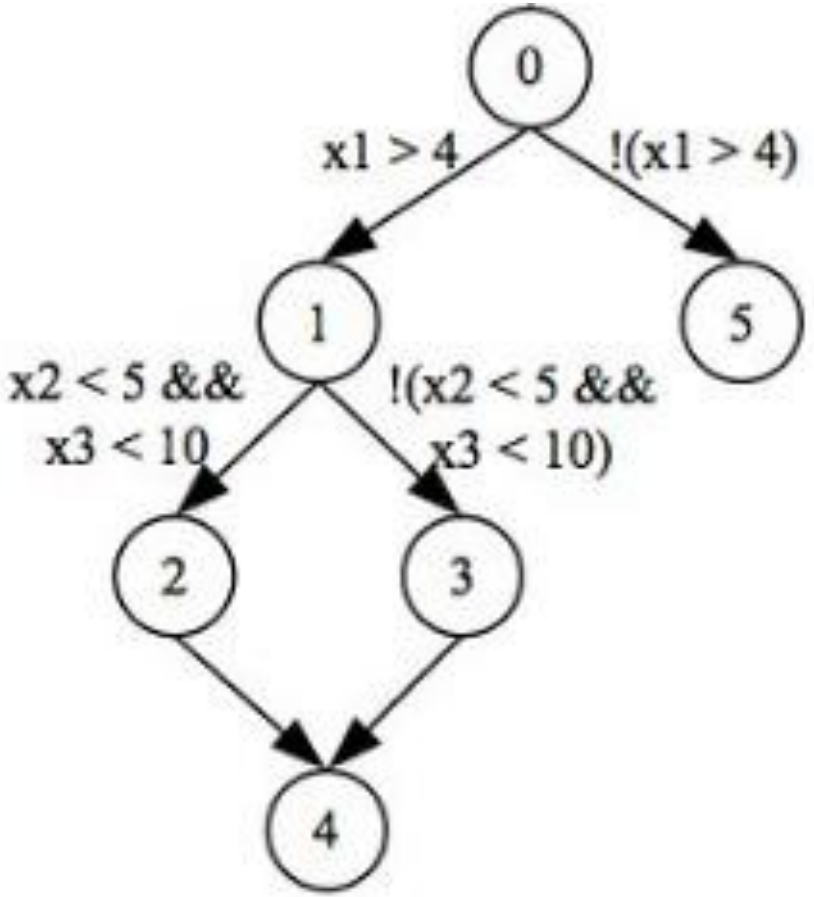
1. V(G) = R, the flow graph has **3** regions.
2. V(G) = E-N+2 = 7 edges – 6 nodes +2 = **3**.
3. V(G) = P+1 = 2 predicate nodes + 1 = **3**



Cyclomatic Complexity

Cyclomatic Complexity

1. V(G) = R, the flow graph has **2** regions.
2. V(G) = E-N+2 = 6 edges – 6 nodes +2 = **2**.
3. V(G) = P+1 = 2 predicate nodes + 1 = **3**



Cyclomatic Complexity: Notations

When to Calculate (The Two Scenarios)

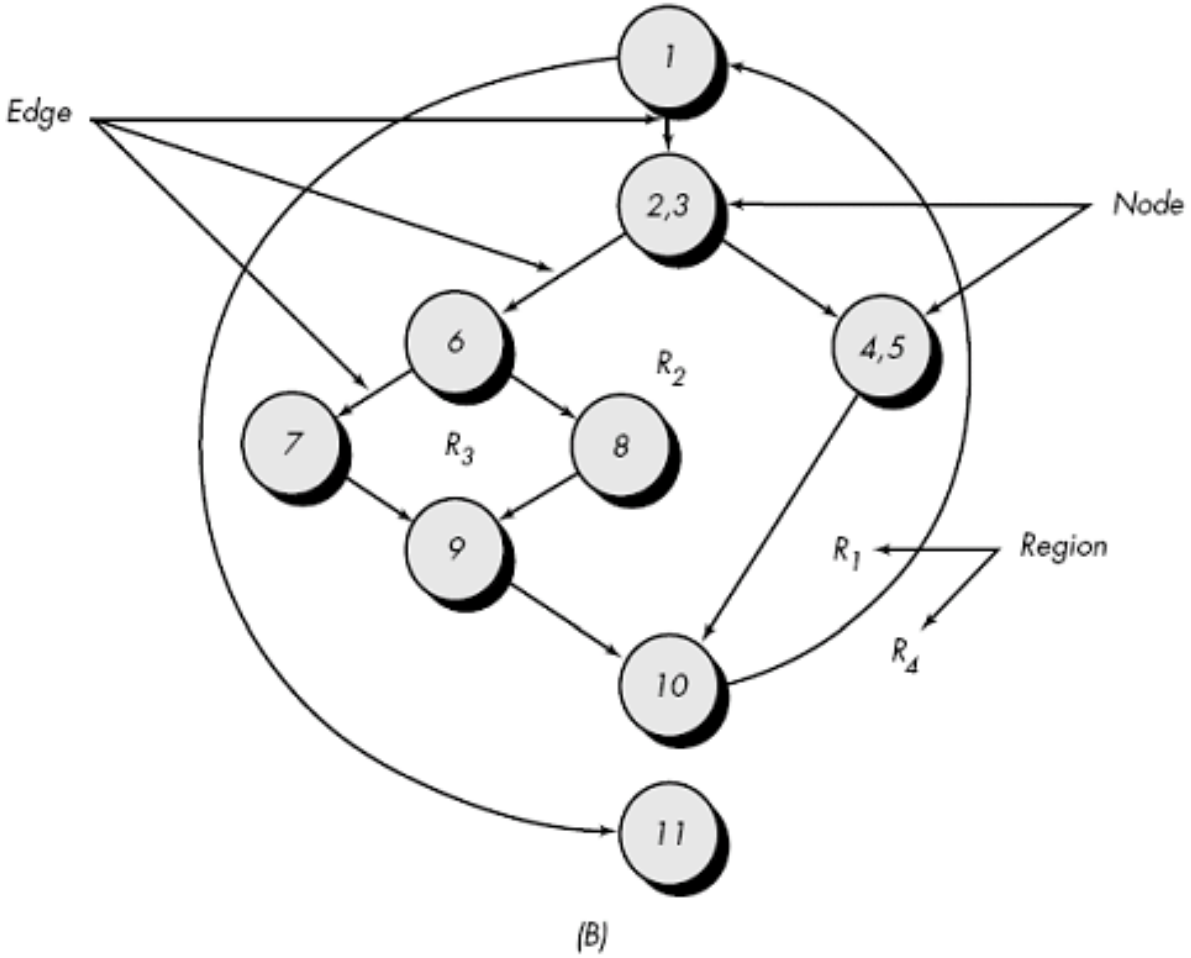
1. **Ideal Code (Planar Graph / *One Exit Node*)**
   - The Control Flow Graph (CFG) is Planar and has a single, clear exit point *(only one return or end point)*. This represents well-structured code .
   - **The Rule: All three formulas agree.**
2. **Complex Code (*Multiple Exit Nodes*)**
   - The CFG has Multiple Exit Nodes *(e.g., multiple return, break, or exit() calls)*.This represents less-structured or complex code.
   - **The Rule: Depend only on the predicate formula.**

Cyclomatic Complexity

Cyclomatic Complexity

1. The flow graph has 4 regions.
2. V(G) = 11 edges – 9 nodes +2 = 4.
3. V(G) = 3 predicate nodes + 1 = 4

Often components with a high V(G) are a high risk for error and should be tested more completely.



White Box Testing Techniques

Basis Path Testing

(1)	(2)	(3)	(4)
Construct the Control Flow Graph(CFG)	Compute the Cyclomatic Complexity Metric(CCM) of the Graph	Identify the Independent Paths	Design Test cases from Independent Paths

Thank You